Instruction format is classified into 5 types based on CPU organisation.

CPU organisation is classified in 3 types based on availability of ALU operands.

(i) Stack-CPU

In this organisation ALU operations are always performed on stack data. that means both of operands are required in stack. After the data manipulation, result is placed on stack only.

stack is a part of memory consists of single entry and single exit through top of stack. (default location) ie TOS (top of stack)

The compatible Instruction format is

[opcode] Zero addr inst format

type of operation

[ALU operation] : source 1 source 2 destn
TOS TOS TOS

pop
pop
ALU opn
push
→ Internal Invocation.

ie TOS ← TOS (ALU opn) TOS

In stack-CPU, each ALU operation pops the IInd operand, pops the Ist operand, operates on them and push result back on stack.

In this organisation, ALU operations are zero addr operation whereas push and pop operation are not a zero addr instruction

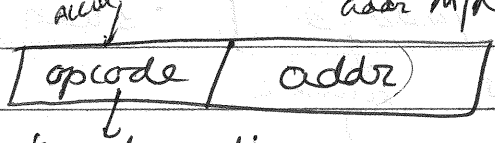
(ii) Accumulator - CPU

- In this organisation, the first ALU operand is always required in accumulator and 2nd ALU operand is present either in register or memory.

- After the data manipulation, result is placed into accumulator.

- ALU is mandatory register in CPU design to hold ALU input and output, therefore it becomes a default location.

- The compatible instruction format is



: one - addr inst format

type of operation.

(iii) General Register - CPU (GPR) : also called classification of ISA machines (Instruction Set Architecture)
This organisation is further divided into 2 types based on no. of register possible in CPU

(a) Register to memory reference CPU

In this structure, processor supports less no. of register. Therefore, register file size is small (maintains the count of GPR in CPU). In this organisation, 1st ALU operand is in register and 2nd operand in register/or memory. After data manipulation, result is placed in SRC1 location.

ALU operation | source 1 reg | source 2 reg/mem | destination source 1

Dt.: Pg.: Delta

destination SRC

| opcode | addr 1 | addr 2 |

: 2 addr Ins format

type of operation

(b) Register to Register CPU

In this architecture. support processor support more no. of register. Therefore register full size is large.

In this Organisation, ALU operation are performed only on registered data that means both of operands are always required in the register. After data manipulation, Result is placed in seperate register.

Load and store are used as data transfer instruction.

| opcode | addr 1 | addr 2 | addr 3 |

: 3 addr ins format

opcode → type of operation → ALU operation

addr 1 → destination → register

addr 2 → source 1 → reg

addr 3 → source 2 → reg.

In this organisation load and store instructions are used to transfer the data b/w memory and register

eg:

| ADD | r_0 | r_1 | r_2 |

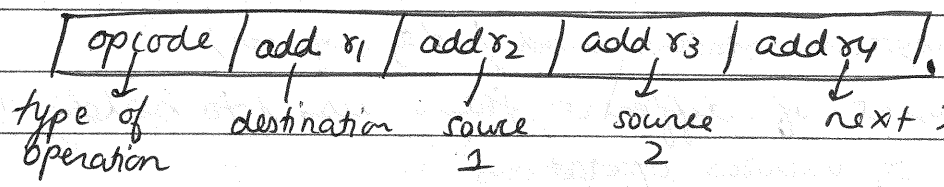
$r_0 \leftarrow r_1 + r_2$

Acc CPU — 8085
reg to mem — 8086 (mov)

Stack CPU only concept not implemented now.

Note: Some of the processor accept both of operands from memory after data manipulation result is placed on register or memory

4-addr instruction format



(I) CPU reads instruction from memory based on PC called instruction fetch.

(II) IR is mandatory register in CPU design. used to hold currently fetched instruction because instruction format is predefined in IR. So it is not a user accessible register.

(III) In the decoding process, respective hardware is enabled in computer to perform the operation.

Operation fetch.

(I) Data is accessed either register or from memory to manipulate called as operand fetch.

Input data is processed based on enabled hardware called processed data.

Dt.:
Pg.:

In CPU design, extra storage is provided ie used to hold the exceptions of operation called as flag reg / program status word.

* Instruction fetch, Instruction decoding, ~~operati~~ operands fetching.
Also known as 2stage pipelining *

PSW (program status word / flag register)
- it consist of different flags used to hold the exceptions of various operations.

- Flag is a flipflop (a bistate component) used to hold one bit ~~operation~~ information

Flag are categorised into two types.

(1) conditional flag. - these are set and reset based on result nature of ALU.
There are of 6 types.

(a) carry flag. - "is there an extra bit out of MSB"

- set = 1 = carry
- reset = 0 = not carry

jmp C
jmp NC

- Carry flag is used in unsigned arithmetic to indicate range exceeding conditions
- in unsigned format, we can represent only +ve data, so sign bit representation not required
- n bit unsigned range is { 0 to $2^n - 1$ }

eg: 4 bit range — 0 to 15

unsigned binary	equivalent decimal
0000	0
0001	1
0010	2

range not exceeding — 0101 + 0110 = X 1011

Carry bit = 0.

1000 + 1001 = (1) 0001

carry bit = 1.

(b) Parity flag — "Is the ALU o/p (Acc) contain even no. of ones
- set = 1 (PE)
- set = 0 (PO)

This flag is used in serial data communication to prepare the error detection code.

(c) Auxiliary carry — "Is there an extra bit from lower nibble (3rd pos) to higher nibble (4th pos).

- set = 1 AC
- set = 0 NAC

This flag is used in BCD arithmetic used to indicate carry exceeding conditions of lower nibble.
BCD format is used to represent the decimal data.
It is of two types.

(i) Packed BCD
(ii) UnPacked BCD — in this format, each decimal digit is represented in 8 bit

	7	6	5	④	③	2	1	0	
eg:	0	0	1	0	0	1	1	0	26
	0	0	1	0	0	1	1	1	27
	0	1	0	0	1	1	0	1	53

no carry from 3 nibble to 4 nibble ∴, so AC = 0.

(d) Zero flag — " is the ALU O/P (acc) contain zero".

- set = 1 = Z
- set = 0 = NZ

This flag is used in control structure implementation on hardware.

(e) Sign flag — " is the MSB Bit of ALU (O/P) (acc) contain

1 —
- set = 1 = NG
- set = 0 = PL

This flag is used to reserve sign of number (MSB bit)

(f) Overflow flag :- "There is carry into MSB and no carry out of MSB or viceversa".

- set = 1 (OV)
- reset = 0 (NV)

	Incarry	outcarry (with refers to MSB)	
EXOR	0	0	NO Overflow
	0	1	
	1	0	overflow
	1	1	

Floating point
- overflow exceptions
- underflow exceptions

signed data - 2's complement (-ve data)

- This flag is used in signed arithmetic to indicate range exceeding condition.

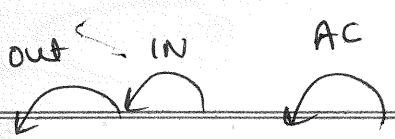
When MSB is zero, then sign is +ve. So, no need to take compliment.

When MSB is 1, sign is -ve. Therefore take two's complement to conclude value.

n bit signed range → $-(2^{n-1})$ to $(2^{n-1}-1)$

4 bit range → $-(2^{4-1})$ to $(2^{4-1}-1)$

-8 to 7 = 16 combinations.



eg:

	7	6	5	4	3	2	1	0	
	1	1	0	0	0	1	0	0	out ✓
+	1	0	1	1	1	0	1	1	In X
(1)	0	1	1	1	1	1	1	1	EXOR operation

carry = 1 parity = 0
sign = 0 (AC) Auxilary carry = 0
OV = 1 Zero flag = 0 (NZ)

Control flags : based on status of these flag, some of operations in hardware level is controlled.

There are of three types.

(a) Trap flag (software generated)
- 0 (g) (go) at a time, program execution
- 1 (t) (trace) single step execution.

(b) Interrupt flag (hardware generated)
- 0 dissable ~~interrupt~~ instruction (DI)
- 1 enable instruction (EI)

flow process wherever sends a signal to CPU that is called interrupt

- maskable interrupt (Generated by low priority prows)
- non-maskable interrupt (generated by high priority prows)

(3) Direction flag — 0 autoIncrement (clear direction)

1 Autodecrement (set direction)

ADDRESSING MODES

It shows the way where exactly object is present in computer.

Object may be Instruction or data.

output of addressing mode is effective address

EA is actual address of object.

Therefore,

object = [EA]

[content of EA]

~~mode field~~

- mode field is used in Instruction Design to specify type of addressing mode used.

- different symbol are used to indicate the various addressing mode.

Dt.:
Pg.:

I) Immediate addressing mode (AM) → #

binding takes place b/w user program and low level program with help of system program

RegAM — Reg AM

[ ] — Direct AM

@ / ( ) — indirect AM

index reg. — indexedreg.

+/- — auto ~~increment~~ indexed (AM)

→ classification of AM

Sequential control flow AM

transfer of control flow AM

Focused on data

Focused on Instruction

Registr ~~name~~ based AM
[EA = Regname]
↓
(i) reg AM

memory based AM
[EA = mem addr]
(i) Implied AM
(ii) Immediate AM
(iii) direct AM
(iv) indirect AM
(v) Indexed AM
(vi) autoindexed AM

[EA = mem addr]
↓
(i) relative PC-relative AM
(ii) Based / Base reg AM.

(1) Sequential control Flow AM

These modes concentrate on location of data in computer. Data may be in two places ie register and memory.

(i) register based AM

(ii) memory based AM.

These modes are used to access data when it is in register. Therefore effective address is Regname.

Under this category only one addressing mode is employed.

1.1) Register Based AM.

These modes are used to access the local variables. In this mode, data is in registers, that register name is present in ADDR field of instruction.

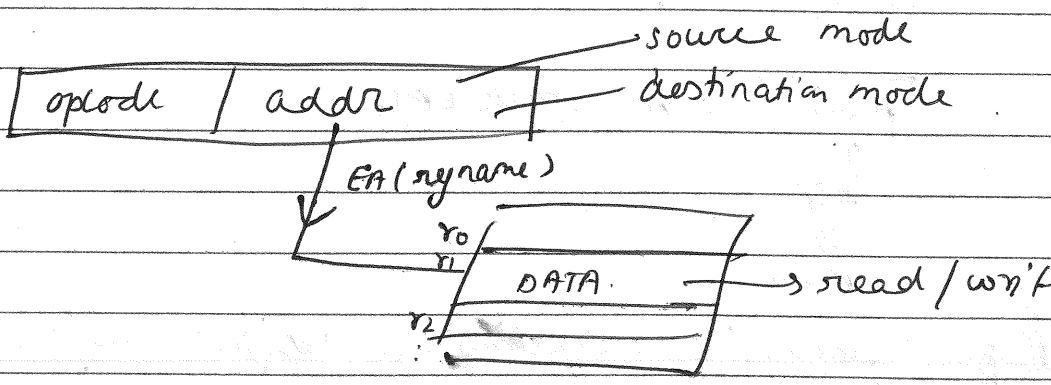
EA = Addr field value (reg name)

Data = content of EA = [EA] = [Regname]

one register reference is required to read/write data. (No need to access the system Bus).

CPU speed = reg speed.

Extra cycles are required to refer memory. ~~Memory reference are costly~~



2) memory based AM.

There modes are used to access data, when it is in memory.
Therfore effective address is memory address.
Under this category different addressing modes are employed.

(a) Implied / Implicit AM.

In this mode, data information is present in opcode itself

zero addr.

opcode

so seperate effective address is not required to access the data.

- type of operation
- data information.

eg: STC → set carry
CY = 1

no need to refer any memory

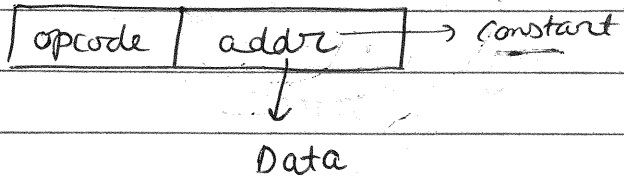
CLC → clear carry.
CY = 0

ADD : on stack CPU
↓ POP
↓ Push

All zero addressing instructions are designed using Implied addressing mode.

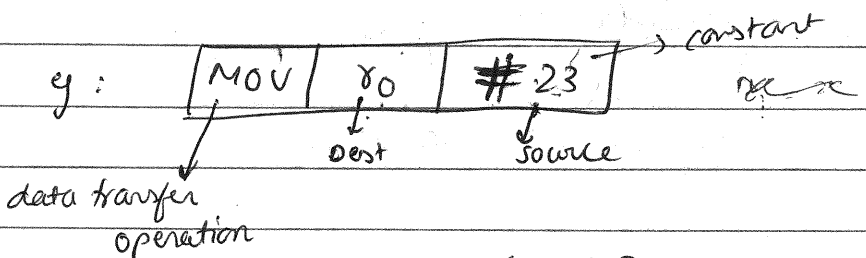
b) Immediate addressing mode. (#)

This mode is used to access the contents. In this mode data is present in addr field of instruction.



$r_0 \leftarrow 23$

data = addr field value ✓



$r_0 \leftarrow 23$.

Limitations

1) The limitations of immediate addressing mode - the range of constant are delimated by size of address field that n bit. Address field supports (0 to $2^n - 1$) Range of unsigned no.

constant signed (-2^{n-1}) to $(2^{n-1} - 1)$

2) Immediate addressing mode is a source addressing mode bcz constants doesnot have storage capability

mov	#23	r0	✗ invalid

Dt.:
Pg.:

c) DIRECT / ABSOLUTE AM [ ]

In this mode, data is in memory that memory address is present in addr field of instruction.

EA = addr field value

↓

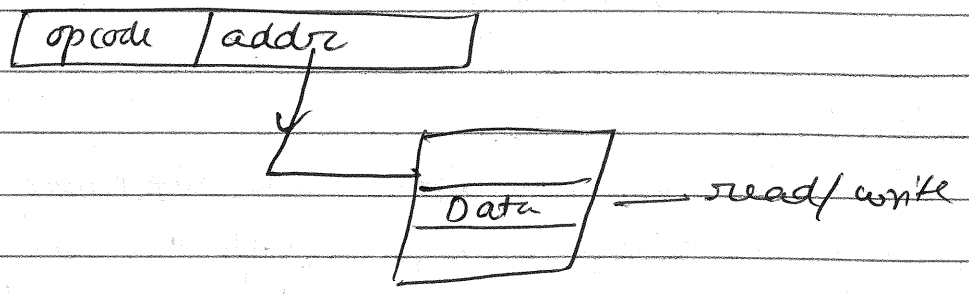
memory value

Data = [EA]

= [mem addr]

memory ref ⇒ To read / write data.

[ ]



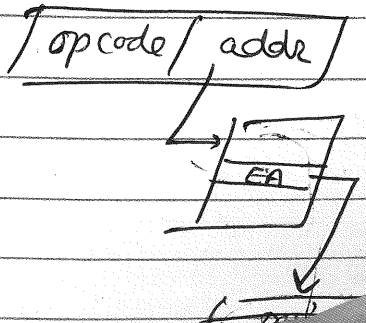
eg: add | r0 | [2000]

→ is address pr jo value hai

$r_0 \leftarrow r_0 + M[2000]$

(d) INDIRECT ADDRESSING MODE

This mode is used to implement pointers. In this mode address field of instruction contain address of effective address.



EA = [addr field value]

Data = [EA]

= [[addr field value]]

EA

This mode is further divided into two types based on availability of effective address.

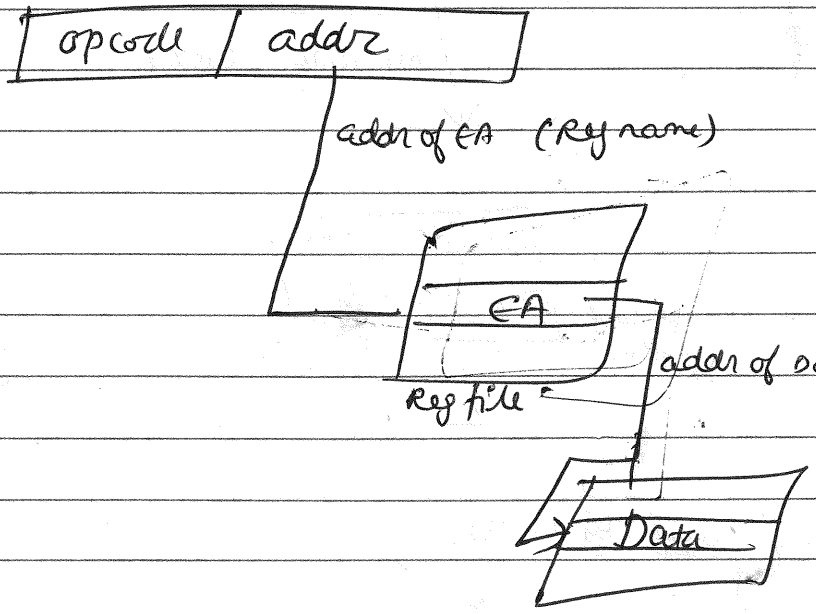
d.1 Reg Indirect AM

In this mode effective address is in register ie register name is present in addr field of instruction.

EA = [addr field value]

↓

regname



Data = [EA] ([ ]).

= [[Regname]]

eg: add | r_0 | @ r_1

↓ regAM ↓ M[[r_1]]

$r_0 \leftarrow r_0 + M[[r_1]]$

Dt.:
Pg.:

4.2 Memory Indirect Addressing mode.

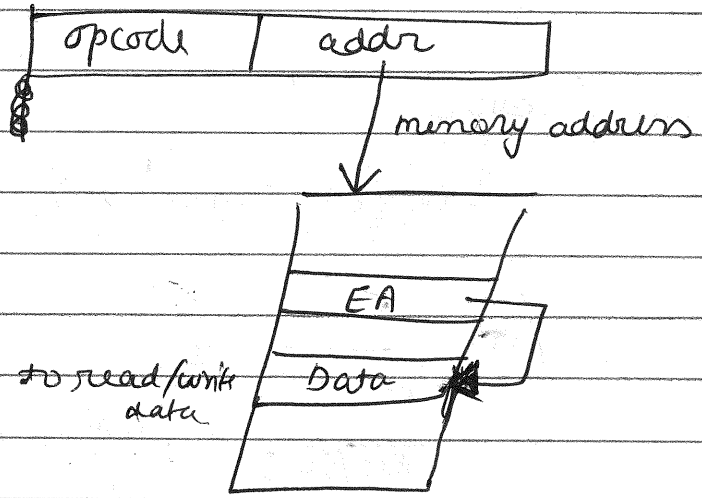
EA = [addr field value]

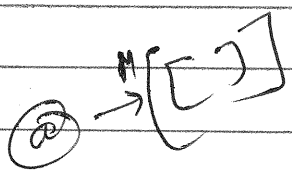
⇓

effective address [mem addr]

Data = [EA]

= [[memory addr]]





eg: | ADD | r_0 | @2000 |

r_0 ← reg AM

@2000 → indirect

M[[2000]]

[[2000]]

$r_0 \leftarrow r_0 + M[[2000]]$

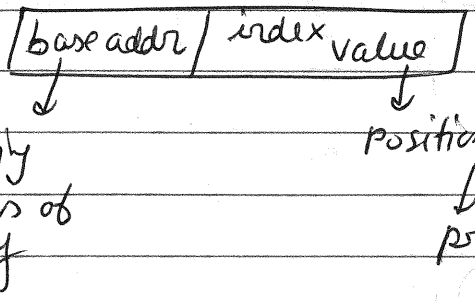
e) INDEXED ADDRESSING MODE

This mode is used to implement arrays. Array is an ordered set of homogeneous data elements.

Array elements are stored in sequential cell .., starting addrss of array called base address

array elements are referred with position called as index.

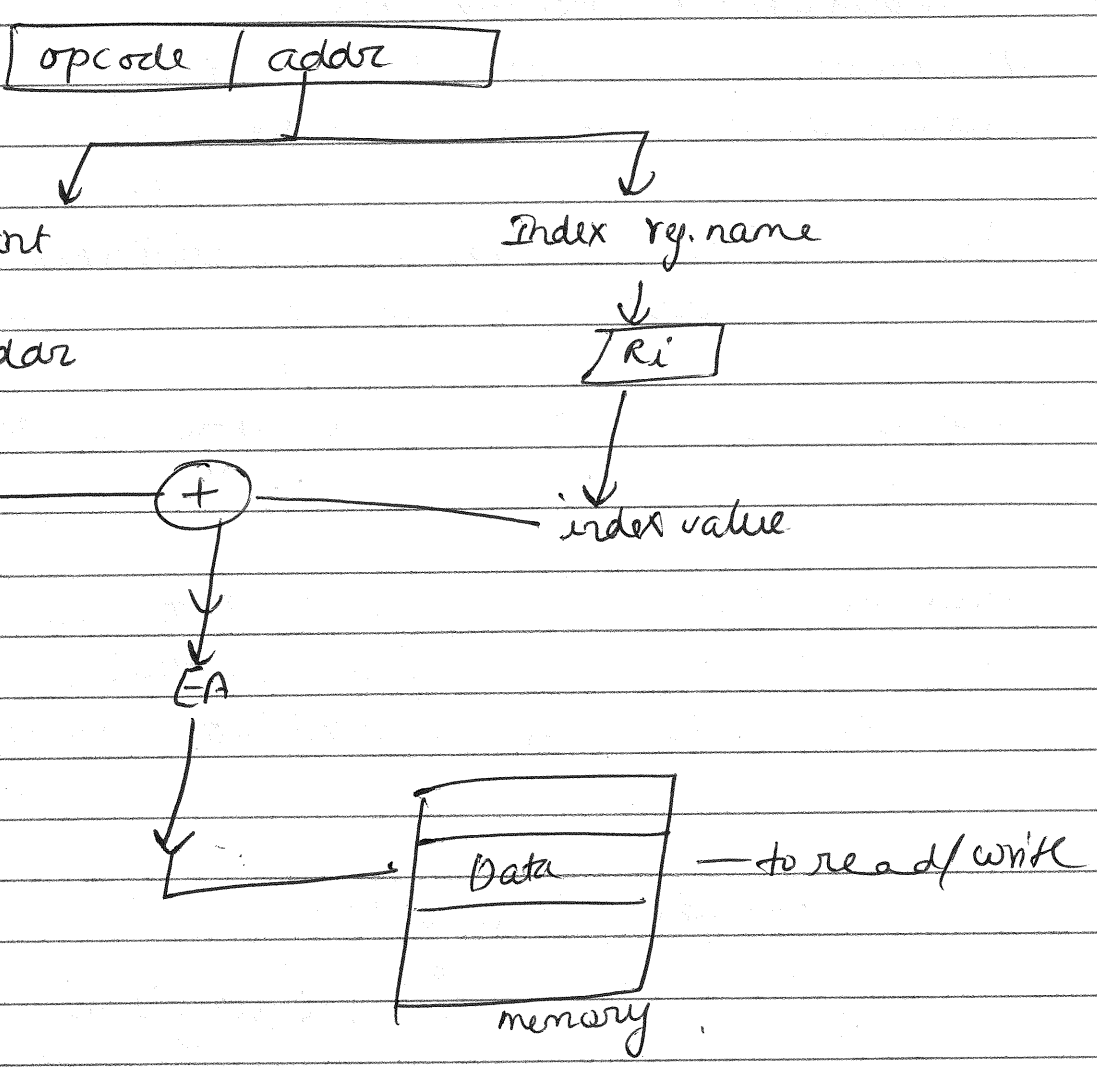
Indexed addressing mode is used to access the array elements from memory. In this mode, EA address is obtained by adding constant value to reg content



$$[EA] = \text{addr field value} + [R_1]$$

$$\text{Data} = [EA]$$

$$[\text{Addr field value} + [R_1]]$$



eg: ADD | r_0 | 3(r_1) → index register

reg AM

$m[3 + [r_1]]$

$r_0 \leftarrow r_0 + M[3 + [r_1]]$

reg ref (write)

reg ref (read)

reg ref (get index)

Note : some of processor support both Base and Index register used to hold base address and index value resp.

Processor with base reg / without base reg.

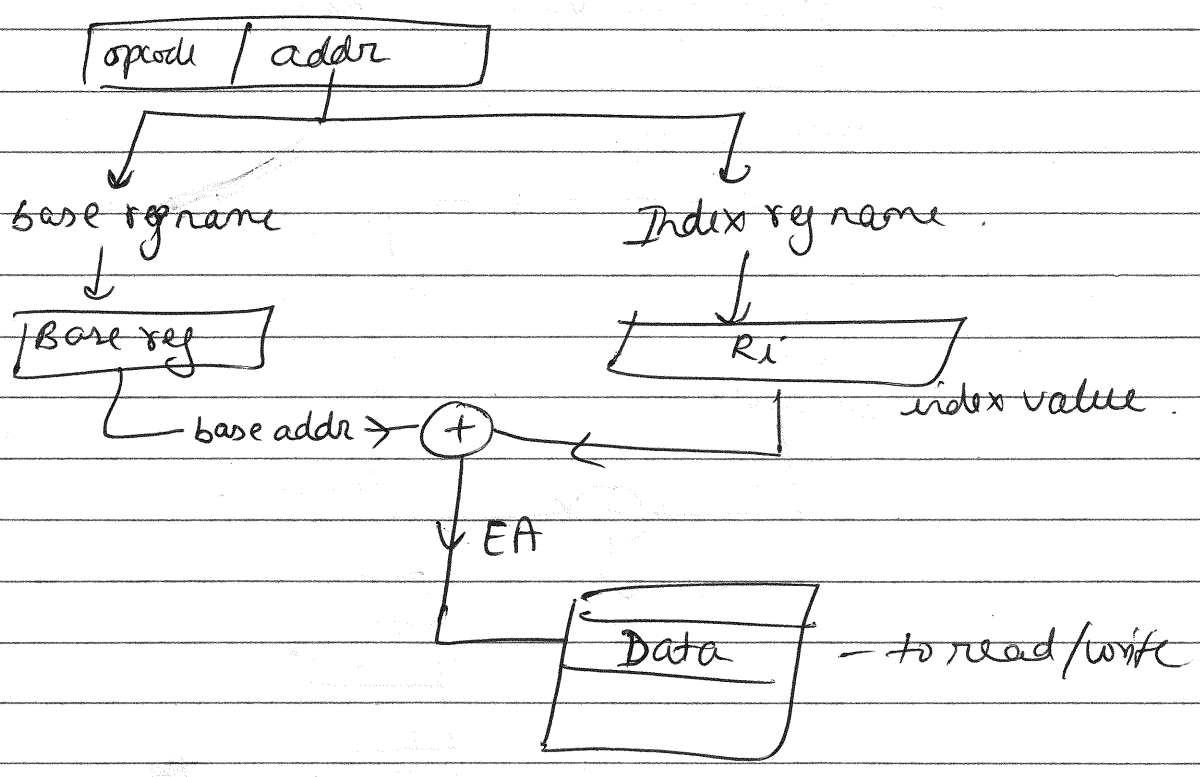
Instruction design changes

In this environment instruction address field hold the base reg name and index reg name.

$$EA = [\text{base reg}] + [\text{index reg}]$$

$$Data = [EA]$$

$$= [[\text{Base Reg}]] + [\text{Index Reg}]$$



eg: | MOV E | AX | [BX] [IR] |

$$AX \leftarrow M[[BX]+[IR]]$$

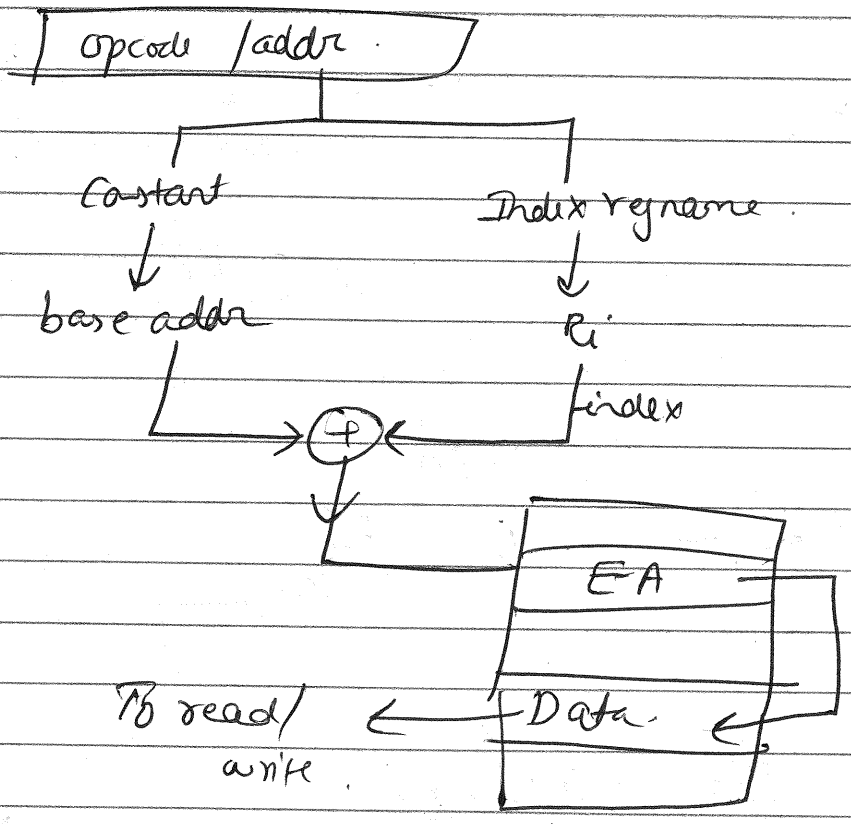
Index addressing mode also known as base addressing mode. This mode is used to access the random array elements from memory

Indirect addressing mode. (takes more time)

In this mode, effective address is present in memory, that memory address is calculated by adding constant value to reg content

$$EA = [\text{addr field value} + [R_i]]$$

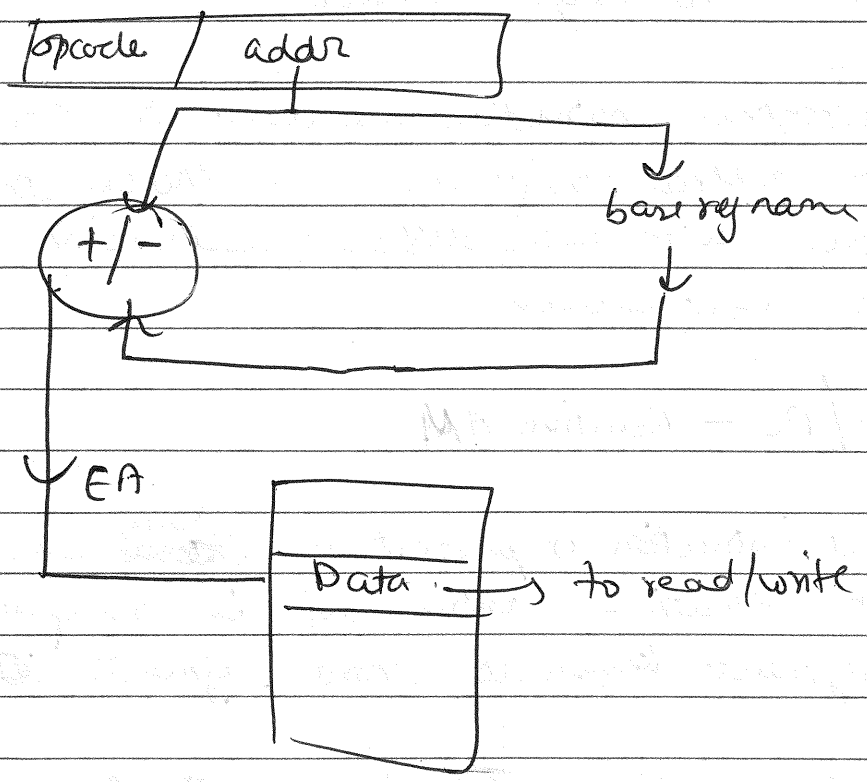
$$\text{Data} = [EA]$$
$$= [[\text{addr field value} + [R_i]]]$$



Dt.:
Pg.:

Auto Indexed addressing mode.

This mode is used to access the linear array elements from array. In this access, the base element is required. In this mode, effective address is calculated either by incrementing / decrementing base adar with step size.



Pre auto increment	MOV	r_0,	$+(r_1)$
Post auto increment	MOV	r_0,	$(r_1)+$
auto pre decrement	MOV	r_0,	$-(r_1)$
post	"	"	$(r_1)-$

r_1 is base address.

→ Transfer of control flow AM – (TOC)

a) These modes concentrate on location of instruction in computer

(ii) when program contain different control structures then during the execution the program control will be transferred to target location

(iii) transfer of control Instructions are used in CPU to implement control structures. So these Instructions are designed with two different addressing modes to calculate target address.

(1) Relative / PC – Relative AM.

When target instruction is present in same segment during program execution control will be transferred to within segment known as Intra segment TOC

* PC relative AM is used to implement the above operation *
In this mode effective address is calculated by adding displacement to PC.
displacement means distance b/w current location to target location.
It is signed constant present in addr field of location

EA = PC + addr field value

Nxt ins addr ← current loc ← Displacement

PC ⇒ PC + Displacement.

(11) Base Reg Am

When target ins is present in seperate segment, then during program execution, control is transferred b/w segment known as Inter segment TOC (transfer of control)

Base addr is different, so Base Reg is used to hold target segment base address. In this mode, effective addr is calculated as

EA = [Base reg] + addr field value.

- EA: next ins addr
- [Base reg]: base addr of target loc
- addr field value: displacement

PC ← [Base Reg] + Displacement

Imp

PC relative or Base Reg both of modes are suitable for program relocation at runtime.

Base reg addressing is best suit to write position independent codes (subprogram)

Segmentation concept was not there consider whole memory chip as one here, then relative value / displacement is applicable.